

Project Status and Completion Report

A roadmap-based assessment of delivered implementation, verification, outstanding requirements, and the practical work needed for a professional launch.

32%

Estimated completion

Full 15-phase roadmap, rounded

37%

Pre-production phases

Phases 0–12 only, rounded

Not yet production-ready

Assessment date: 5 October 2026

Assessment basis: Helpyard.store Professional Developer Implementation Plan dated 4 October 2026, current repository source, migrations, README, and available tests.

Repository: ofriday0420-code/helpyard.store

Prepared as an engineering status estimate. It is not an audit certificate, security certification, production acceptance, or independent verification of live database/payment behavior.

1. Executive summary

Estimated total roadmap completion is approximately 32%. The web storefront, shared catalog, customer authentication, cart, checkout/order creation, an SSLCOMMERZ payment integration, initial physical-order fulfillment, and the first protected-download foundation are represented in code. Several important capabilities are still partial, unconfigured, or not end-to-end verified.

The plan has 15 phases (0–14), but specifies no phase weighting or percentage method. For transparency, this report assigns a practical completion estimate to each phase, averages those phase percentages equally, and rounds the result: $475 \text{ percentage-points} \div 15 \text{ phases} = 31.7\%$, reported as **32%**. This is a directional estimate, not a precise measure of elapsed effort, feature count, or business value. Different effort weights could reasonably move the estimate several percentage points.

Across phases 0–12, which cover groundwork through production deployment, the comparable average is $475 \div 13 = 36.5\%$, reported as **37%**. This does not mean the release is 37% launch-ready: missing production gates such as live database testing, verified payments, digital/course/software fulfillment, complete admin operations, security/backup testing, and deployment remain blockers.

Important verification limit: the active PHP CLI reports no PDO database driver. Automated tests pass, but database migrations, transactional checkout/payment/download entitlement flows, and ownership constraints have not been exercised against MySQL/MariaDB in this environment. SSLCOMMERZ sandbox and production merchant credentials/verification are also not evidenced.

Current delivery position

- **Working code foundation:** PHP MVC-style organization, configuration, migrations, seed data, storefront/catalog views, web routes, and a small JSON product/category API.
- **Core commerce slices:** customer accounts, addresses, session cart, server-side checkout calculations, order snapshots, stock reservation and cleanup, and payment callback validation logic.
- **Early fulfillment:** staff-only physical/book status workflow and shipment tracking, plus private-file metadata, entitlement creation after validated payment, and customer download handler.
- **Major launch scope:** full admin operations, course learning/enrollment, software licensing, upload/file administration, email verification/reset/notifications, complete API contracts, comprehensive QA, production deployment, and operations runbooks.

2. Phase-by-phase completion

Scores reflect source and test evidence at the report date. A phase is not considered fully accepted merely because part of its code exists; missing dependencies or unexecuted acceptance checks reduce its score.

Plan phase	Estimate	Delivered / evidence	Still needed
0 — Requirements & wireframes	0%	No approved requirements package, wireframes, or acceptance backlog found.	Confirm scope, product and pricing rules, refund/shipping policy, roles, user flows, wireframes, acceptance criteria.
1 — Engineering foundation	70%	MVC-style PHP code, configuration, migrations/seeder, test runner, setup README, Git history.	Validate supported environments; add operational logging/error monitoring, robust migration/restore process, dependency/version and deployment setup.
2 — UI/UX system	45%	Responsive storefront pages and shared view components.	Approved design system, reusable full component set, mobile navigation and formal 320/375/768/1024/desktop checks; accessibility review.
3 — Catalog & product engine	55%	Five section routes, shared product catalog, search/filter API, details, images and variants.	Complete specifications, SEO fields, demos/files metadata management, and distinct business rules for all product types.
4 — Accounts & dashboard	40%	Registration, login/logout, profiles, saved addresses, sessions, throttling, recent customer orders.	Email verification, password reset, customer roles/permissions, wishlists, notification center, integrated downloads/courses/licenses.
5 — Cart & checkout	65%	Database cart, server-side price/stock validation, address ownership, order/address/item snapshots, inventory reservation.	Coupons, shipping rules/calculation, cart merge, robust duplicate submission/concurrency integration tests and complete failure behavior.
6 — Payments	60%	SSLCOMMERZ initiation, provider validation of browser return/IPN, transaction records, duplicate/risk/review handling.	Sandbox transactions and failure/retry acceptance, merchant setup, production-safe configuration and reconciliation operations.
7 — Fulfillment	30%	Physical/book staff transitions and tracking; private file/entitlement schema, verified-payment entitlement creation, guarded customer download path.	Database verification, operator file management, limits/revocation workflow, course enrollment/learning, software license assignment, fulfillment email.
8 — Admin panel	10%	Only a focused staff fulfillment queue exists; admin role is checked on it.	Dashboard, catalog/category, orders, payments/review, inventory, customers, coupons, content, support, reports, audit logs, admin access lifecycle.
9 — Search & SEO	20%	Clean product URLs and catalog search/filter.	Titles/descriptions, canonical URLs, sitemap, robots.txt, Open Graph, structured data and internal-link verification.
10 — Security hardening	50%	Prepared statements, CSRF, output escaping, session rotation, login throttling, ownership checks, private-path checks, verified payment validation.	Independent security tests, secure cookie/HTTPS production settings, secure upload, admin audit controls, backup policy, incident/error monitoring and high-risk review.

Plan phase	Estimate	Delivered / evidence	Still needed
11 — QA & acceptance	30%	Automated PHP checks cover environment/SQL parsing, route behavior, validation and selected policies; local test suite passes.	MySQL integration suite, checkout/payment sandbox tests, cross-user/admin isolation, browser/mobile/accessibility, performance/load and backup-restore matrix.
12 — Production deployment	0%	No verified production deployment or smoke-test evidence found.	Hosting, DNS/TLS, secrets, DB migration, file permissions, scheduler, monitoring, backups, rollback and deployment runbook.
13 — PWA	0%	Not started; correctly deferred pending stable web/API platform.	Manifest, service worker, offline shell, installability, push policy and PWA acceptance tests.
14 — Native mobile application	0%	Not started; explicitly deferred by roadmap.	Stable versioned APIs and auth contract first; then Flutter/React Native app, secure tokens, release/build/test process.

3. What is currently implemented

Storefront and catalog

- Responsive storefront home and routes for Courses, Ready Websites, Official Software, Books, and Security Devices.
- Shared product/category schema and catalog/product APIs with search/filtering; product detail includes active images/variants.
- Seed products and categories; MySQL/MariaDB is expected for database-backed operation.

Accounts and commerce

- Customer registration/login/logout, password hashing, CSRF-protected mutations, session rotation, login attempt throttling, profile and saved address management.
- Session-keyed database cart with database price/stock checks, authenticated checkout, transaction-backed stock reservation, address/product snapshots, order view, and reservation cleanup CLI.
- SSLCOMMERZ hosted payment code is credential-gated; server-side provider validation is used before payment confirmation.

Fulfillment in progress

- Staff fulfillment queue, role checks, CSRF protection, constrained physical/book order transitions and carrier/tracking fields.
- Private storage location defaults to `storage/private`; override with `PRIVATE_STORAGE_PATH`. Files should be kept outside the public web root.
- Entitlement rows are created transactionally when a validated, low-risk payment moves the order to paid. Customer route filters by ownership, entitlement state, active metadata, and eligible order status before serving attachment content.

Operational setup still required: downloadable products have no admin upload/file-management interface. A trusted operator must place files in the configured private store and register matching metadata. The DB migration and payment-to-entitlement transaction require MySQL integration verification before use.

4. Priority future work

Priority	Workstream	Definition of an efficient completion
P0 · Immediate gate	Database and integration confidence	Provision repeatable MySQL 8/MariaDB test DB with pdo_mysql enabled; apply migrations 001–011 from clean state; test payment confirmation and duplicate callbacks, entitlement creation, file ownership and revocation, shipment transitions and concurrent stock safety.
P0 · Immediate gate	Requirements decisions	Approve product types, file/download policy, download expiry/limits, refunds/revocation, course rules, license assignment, shipping zones/pricing, email provider, roles and support operations before scaling features.
P1	Admin foundation and private files	Provision admin securely; implement role/permission checks and audited catalog/order/file management; validate file size/type, random storage names, safe private path, malware handling decision, replacement/revocation and backups.
P1	Course first-release flow	Course/section/lesson tables, paid enrollment idempotency, access controls, lesson progress, customer course screen and test cases; do not expose media publicly.
P1	Payment and fulfillment validation	Run SSLCOMMERZ sandbox end-to-end including fail/cancel/retry/IPN/duplicate/late/risky callbacks; define payment review and refunds before production credentials.
P1	Email and customer lifecycle	Select provider; implement verified email, password recovery and queued payment/order/shipment/course notifications with observable retries and no lost order transitions.
P2	Licensing and ready-website handoff	Agree license source/uniqueness/assignment, activation, revocation and support policy; implement auditable one-time assignment and customer access without leaking reusable secrets.
P2	SEO and accessibility	Add metadata, canonical/sitemap/robots/structured data; test semantic navigation, focus, contrast, keyboard operation and mobile/responsive acceptance.
P2	Production readiness	Deploy staging; HTTPS and hardened cookies; secret management; restore-tested backups; logs/alerts; scheduler; rollback; incident/support ownership; signed launch checklist.
P3 · Later	PWA then native client	Stabilize documented/versioned REST APIs first; ship installable PWA, then consider native Flutter/React Native only after shared backend flows pass acceptance.

5. Professional requirements and decisions

People, policy and ownership

- Name a product owner to approve pricing, shipping, delivery, refund, coupon, licensing, course access, and support rules.
- Assign a technical owner for releases, DB migrations, secrets, backups, monitoring, incident response and payment reconciliation.
- Provision administrator accounts only through controlled operations; no public promotion/registration to admin.
- Identify authorized content/file owners, rights to distribute digital assets, and a review process for uploaded files.

Environments and services

- Separate development, staging and production database/credentials. Enable and verify PHP 8.3+ with `pdo_mysql`; select supported MySQL 8 or MariaDB version and pin PHP/package dependencies.
- Obtain SSLCOMMERZ merchant account and sandbox credentials; configure public HTTPS callback/IPN URLs only in controlled environments. Define refund/chargeback reconciliation and support ownership.
- Select transactional email provider/domain and configure SPF/DKIM/DMARC, bounce visibility, secrets, retry policy and template ownership.
- Choose hosting/runtime, TLS/DNS, private object/file storage, backup retention, restore target, uptime/logging provider and alert recipients.

Engineering quality gates

- Use branch/PR review, protected production branch, small reviewed changes, code style/static checks, migration review and reproducible releases.
- Build integration tests for stateful DB behavior and acceptance tests for cross-customer access, admin access, payment idempotency, inventory contention, entitlement lifecycle and private-file path checks.
- Run dependency/security scanning, secrets checks, upload/path validation tests, authorization review, HTTPS/cookie checks and incident-response exercise.
- Maintain setup, API contract, admin/customer guide, payment operations, deployment/rollback, backups/restore, test report and launch checklist.

Launch rule: do not advertise paid downloads, course access, licensing, shipping, or live payment as operational until their real integration and access-control acceptance tests pass in staging and an accountable owner signs off.

6. Verification performed and limits

- **Automated test command:** `php tests/run.php` — passed at the current implementation checkpoint; includes migration parsing, route/auth guards, selected checkout/payment validation, fulfillment policies, and private download path/response checks.
- **Syntax check:** PHP lint across application, routes, tests, migrations/CLI and config — passed.
- **Whitespace check:** `git diff --check` — passed.
- **Runtime limitation:** PHP 8.3.33 was present, but no PDO database driver was listed. The relational database integration cannot run in this environment.
- **External services:** no live MySQL/MariaDB verification, SSLCOMMERZ sandbox payment, email delivery, production deployment, backup restore, or third-party security assessment is evidenced.
- **Repository state:** implementation includes files currently modified/untracked in the working tree; this report assesses the local source snapshot, not a clean deployed release.

Suggested next acceptance run

1. Enable `pdo_mysql`, provision disposable MySQL 8/MariaDB test database, apply all migrations from clean state and seed sample catalog.
2. Add non-public sample file metadata; prove no anonymous/cross-user/unpaid/review/refunded/revoked access; verify the authorized user's bytes and headers.
3. Run sandbox checkout with success, cancellation, failure, repeated callbacks, risk-review and expired reservation cases; assert stock/payment/order/entitlement state after every case.
4. Exercise physical fulfillment transitions and confirm customer shipment visibility is limited to the order owner.
5. Record output, environment versions, test failures/remediation and sign-off in the release test report.

7. Conclusion

Helpyard.store has progressed beyond a visual prototype: substantial storefront and first commerce flows exist, and a foundation for two fulfillment types has begun. The most accurate concise rating is **about 32% complete across the entire 15-phase roadmap**, or about **37% through the pre-production phases** by the equal-phase estimate used here.

The project should currently be treated as a development-stage implementation, **not ready for production commerce**. Efficient progress now depends on enabling a database-backed integration environment and agreeing operational/product rules while completing admin, course/licensing/email flows and launch QA in small, acceptance-tested increments. PWA and native mobile should remain deferred until the shared web platform/API are stable.

Primary source: Helpyard.store Professional Developer Implementation Plan, dated 4 October 2026, phases 0–14 and production definition of done. Repository evidence: current README, source routes/controllers/repositories/services, migrations 001–011, and tests/run.php.